

# Guía de Aprendizaje: Funciones en C

## Monitorías de Sistemas Operativos

### 1. Motivación

Construir un Sistema Operativo completo en un solo bloque de código sería imposible. Las funciones nos permiten dividir problemas grandes en partes pequeñas, reutilizables y fáciles de probar.

### 2. Idea Fundamental

Una función es un bloque de código independiente que realiza una tarea específica y puede devolver un resultado.

### 3. Sintaxis Básica

```
tipo_retorno nombre_funcion(parametros){  
    // Cuerpo  
    return valor;  
}
```

### 4. Ejemplo Mínimo Funcional

```
1  #include <stdio.h>  
2  
3  int cuadrado(int n){  
4      return n * n;  
5  }  
6  
7  int main(){  
8      int x = 5;  
9      printf("El cuadrado es %d\n", cuadrado(x));  
10     return 0;  
11 }
```

### 5. Explicación Paso a Paso

1. Se define la función `cuadrado` que recibe un entero y devuelve un entero.
2. En `main`, se llama a la función pasando `x` como argumento.

3. La ejecución salta a la función, calcula el resultado y lo devuelve.
4. El valor devuelto se usa en el `printf`.

## 6. Qué ocurre en memoria

Cada vez que se llama a una función, se crea un **Stack Frame** en la memoria Stack. Allí se guardan los parámetros y las variables locales. Al terminar la función, ese espacio se libera automáticamente.

## 7. Patrones comunes de uso

Modularización para cálculos repetitivos:

```
1 float calcular_promedio(int suma, int cantidad){  
2     return (float)suma / cantidad;  
3 }
```

## 8. Errores Comunes

- **Variables locales inaccesibles:** Intentar usar una variable de una función desde otra.
- **Falta de prototipo:** Llamar a una función antes de definirla sin haber declarado su prototipo arriba.

## 9. Buenas Prácticas

- Una función debe hacer solo una cosa (*Single Responsibility Principle*).
- Usar verbos para los nombres de las funciones (ej: `ordenar_arreglo`).

## 10. Ejercicios

1. Escriba una función que reciba el radio de un círculo y devuelva su área.
2. Cree una función que reciba un número e imprima si es par o impar (sin devolver valor).